ELEC6234 – Embedded Processors

Coursework Report – picoMIPS implementation

Yan Haien
hy5n25
Microelectronics Systems Design (MSc)
Tomasz Kazmierski

1. Introduction

State what you achieved in this assignment. Summarise briefly your preparation work, your experimental work, and results. Specifically, state which parts of the assignment were delivered according to the requirements and summarise any extensions to the basic specification you have carried out with references to the sections. (approx. 0.5 page).

Design Details Form			
Total Cost: 72	**ALMs: 72**	**Memory bits: 0**	**Multipliers:1**

Flow Status	Successful - Fri Apr 24 12:47:28 2026
Quartus Prime Version	22.1std.2 Build 922 07/20/2023 SC Standard Edition
Revision Name	picoMIPS4test
Top-level Entity Name	picoMIPS
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	72 / 32,070 (< 1 %)
Total registers	23
Total pins	19 / 457 (4 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	1 / 87 (1 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Fig 1.1 Summary report of hardware cost for the picoMIPS implementation on Cyclone V SoC 5CSEMA5F31C6.

Instruction Set

NOP: No operation is performed. Writing to general-purpose registers is disabled. Execution time is one clock cycle.

ADD: Computes the sum of the values in register Rd and Rs, and overwrites the Rd register with the result. No immediate value is involved. Execution time is one clock cycle.

MULI: Multiplies the value in register Rs by a 2's complement immediate value, and overwrites the register Rd with the result. Execution time is one clock cycle.

LOADI: Computes the ROM address by adding a 2's complement immediate value to the value in Rs register, and overwrites the Rd register with the data read from this address. Execution time is one clock cycle.

Instruction Format

Table 1.1 Instruction Format

12 bits	11-10 (opcode)	9-8	7-6	5-0
NOP	00	00	00	000000
ADD	01	Rd	Rs	(Ignored)
MULI	10	Rd	Rs	Imm
LOADI	11	Rd	Rs	Imm

Program

```
// total 16 lines of instructions
// format: 2b opcode, 2b %d, 2b %s, 6b Imm
// ADD: no Imm

// HEX // BINARY                 // ASSEMBLER          // DISCRIPTION
000    // 12'b00_00_00_000000 // NOP

F7E    // 12'b11_11_01_111110 // LOADI   %3, %1, -2   // LOAD W[i-2]          to R3
BCA    // 12'b10_11_11_001010 // MULI    %3, %3, 10   // MUL   W[i-2]*K[0]    to R3

E7F    // 12'b11_10_01_111111 // LOADI   %2, %1, -1   // LOAD W[i-1]          to R2
A95    // 12'b10_10_10_010101 // MULI    %2, %2, 21   // MUL   W[i-1]*K[1]    to R2
780    // 12'b01_11_10_000000 // ADD     %3, %2       // ADD   W*K+R3         to R3

E40    // 12'b11_10_01_000000 // LOADI   %2, %1, 0    // LOAD W[i]            to R2
A9B    // 12'b10_10_10_011011 // MULI    %2, %2, 27   // MUL   W[i]*K[2]      to R2
780    // 12'b01_11_10_000000 // ADD     %3, %2       // ADD   W*K+R3         to R3

E41    // 12'b11_10_01_000001 // LOADI   %2, %1, +1   // LOAD W[i+1]          to R2
A95    // 12'b10_10_10_010101 // MULI    %2, %2, 21   // MUL   W[i+1]*K[3]    to R2
780    // 12'b01_11_10_000000 // ADD     %3, %2       // ADD   W*K+R3         to R3

E42    // 12'b11_10_01_000010 // LOADI   %2, %1, +2   // LOAD W[i+2]          to R2
A8A    // 12'b10_10_10_001010 // MULI    %2, %2, 10   // MUL   W[i+2]*K[4]    to R2
780    // 12'b01_11_10_000000 // ADD     %3, %2       // ADD   W*K+R3         to R3

000    // 12'b00_00_00_000000 // NOP
```

Design Block Diagram

2. Overall architecture of the design and simulations

Describe your processor architecture , showing the Control Path and Data Path. You can show snippets of your System Verilog source code. There is no need to show the full source code for all your modules in the report as the full source code must be submitted separately. ***Do not copy any code or diagrams from the lectures.*** *Give your Modelsim testbenches and Modelsim results.* ***DO NOT make statements such as: "Figure 2 shows the simulation results of the module functioning correctly".*** *Instead, explain the results shown in the figures to demonstrate that you understand how the tested modules work. . (max 2 pages).*

3. FPGA implementation

Explain how you tested your design after programming the FPGA. In case you had to edit your original code and re-synthesize – explain what you did. (approx. 1-2 pages)

4. Conclusion

Give your general conclusion, comment on what you learnt. Comment on ways to improve the design or extend it further. (approx.0.25 – 0.5 of a page)

5. References

Quote the sources of your information. Especially make reference to any sources you used in the development of your code.